

Springboot Backend connection with S3 bucket

26 July 2026 12:05

Aim:

- 1.) Create Springboot APP
- 2.) AWS SDK (copy the code from maven repository)
- 3.) S3 Connection with local APP
- 4.) Upload/read file in S3 bucket

Main Concept:

From Local Springboot application->AWS SDK->S3 client

Step1: Create Springboot App

Step2: After opening the Eclipse IDE you have to create a project from start.spring.io and the you have to upload the Amazon aws sdk from maven repository and paste the code in the pom.xml.
code from maven repository:

```
Search amazon awssdk:
<!-- Source: https://mvnrepository.com/artifact/software.amazon.awssdk/s3 -->
<dependency>
  <groupId>software.amazon.awssdk</groupId>
  <artifactId>s3</artifactId>
  <version>2.47.5</version>
  <scope>compile</scope>
</dependency>
```

Step3: Create a format

```
project_name
|
|-- config
|   |-- S3Config.java
|
|-- controller
|   |-- S3Controller.java
|
|-- service
|   |-- S3Service.java
|   |-- S3ServiceImpl.java
|
|-- ProjectApplication.java
```

S3Config.java:

```
package com.SpringBootBackendWithS3.S3ConnectionDemo.config;

import org.springframework.beans.factory.annotation.Value;
import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;

import software.amazon.awssdk.auth.credentials.AwsBasicCredentials;
import software.amazon.awssdk.auth.credentials.AwsCredentials;
import software.amazon.awssdk.auth.credentials.StaticCredentialsProvider;
import software.amazon.awssdk.regions.Region;
import software.amazon.awssdk.services.s3.S3Client;

@Configuration
public class S3Config {

    @Value("${cloud.aws.credentials.access-key}")
    private String accessKey;

    @Value("${cloud.aws.credentials.secret-key}")
    private String secretKey;

    @Value("${cloud.aws.region.static}")
    private String region;

    @Bean
    public S3Client s3Client() {
        AwsCredentials awsBasicCredentials =
            AwsBasicCredentials.create(accessKey, secretKey);

        return S3Client.builder()
            .region(Region.of(region))
            .credentialsProvider(StaticCredentialsProvider.create(awsBasicCredentials))
            .build();
    }
}
```

S3Service.java:

```
package com.SpringBootBackendWithS3.S3ConnectionDemo.service;

import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.beans.factory.annotation.Value;
import org.springframework.stereotype.Service;
import org.springframework.web.multipart.MultipartFile;
import software.amazon.awssdk.core.ResponseBytes;
import software.amazon.awssdk.core.sync.RequestBody;
import software.amazon.awssdk.services.s3.S3Client;
import software.amazon.awssdk.services.s3.model.GetObjectRequest;
import software.amazon.awssdk.services.s3.model.GetObjectResponse;
import software.amazon.awssdk.services.s3.model.PutObjectRequest;

import java.io.IOException;

@Service
public class S3Service {

    @Autowired
    private S3Client s3Client;

    @Value("${aws.bucket.name}")
    private String bucketName;

    public void uploadFile(MultipartFile file) throws IOException {
        s3Client.putObject(PutObjectRequest.builder()
            .bucket(bucketName)
            .key(file.getOriginalFilename())
            .build(),
            RequestBody.fromBytes(file.getBytes()));
    }

    public byte[] downloadFile(String key) {
        ResponseBytes<GetObjectResponse> objectAsBytes =
            s3Client.getObjectAsBytes(GetObjectRequest.builder()

```

```

        .bucket(bucketName)
        .key(key)
        .build());
    return objectAsBytes.asByteArray();
}
}

```

S3Controller.java

```

package com.SpringBootBackendWithS3.S3ConnectionDemo.controller;

import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.http.HttpHeaders;
import org.springframework.http.MediaType;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.*;
import org.springframework.web.multipart.MultipartFile;

import com.SpringBootBackendWithS3.S3ConnectionDemo.service.S3Service;

import java.io.IOException;

@RestController
public class S3Controller {
    @Autowired
    private S3Service s3Service;

    @PostMapping("/upload")
    public ResponseEntity<String> upload(@RequestParam("file") MultipartFile file) throws IOException, IOException {
        s3Service.uploadFile(file);
        return ResponseEntity.ok("File uploaded successfully!");
    }

    @GetMapping("/download/{filename}")
    public ResponseEntity<byte[]> download(@PathVariable String filename) {
        byte[] data = s3Service.downloadFile(filename);
        return ResponseEntity.ok()
            .header(HttpHeaders.CONTENT_DISPOSITION,
                "attachment; filename=\"" + filename + ".pdf\"")
            .contentType(MediaType.APPLICATION_PDF)
            .body(data);
    }
}

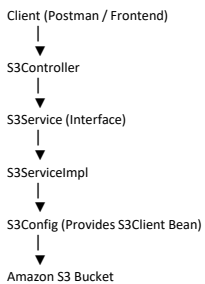
application.properties:

spring.application.name=S3ConnectionDemo
server.port=4566
cloud.aws.credentials.access-key=access-key
cloud.aws.credentials.secret-key=secret-key
cloud.aws.region.static=ap-south-1

aws.bucket.name=bucket-name

```

Step4: Go to iam->iam users-> the user you have click on it-> create access key work flow:



Step5:

copy the access key and secret key and paste them in the application.properties

Step6: Create an S3 bucket and create a folder if you want else dont create it , copy the name of the bucket and paste it into the application-properties

In the region section copy the ap-south-1 and paste it into the application.properties else wont recognize where the request is coming from will cause more latency

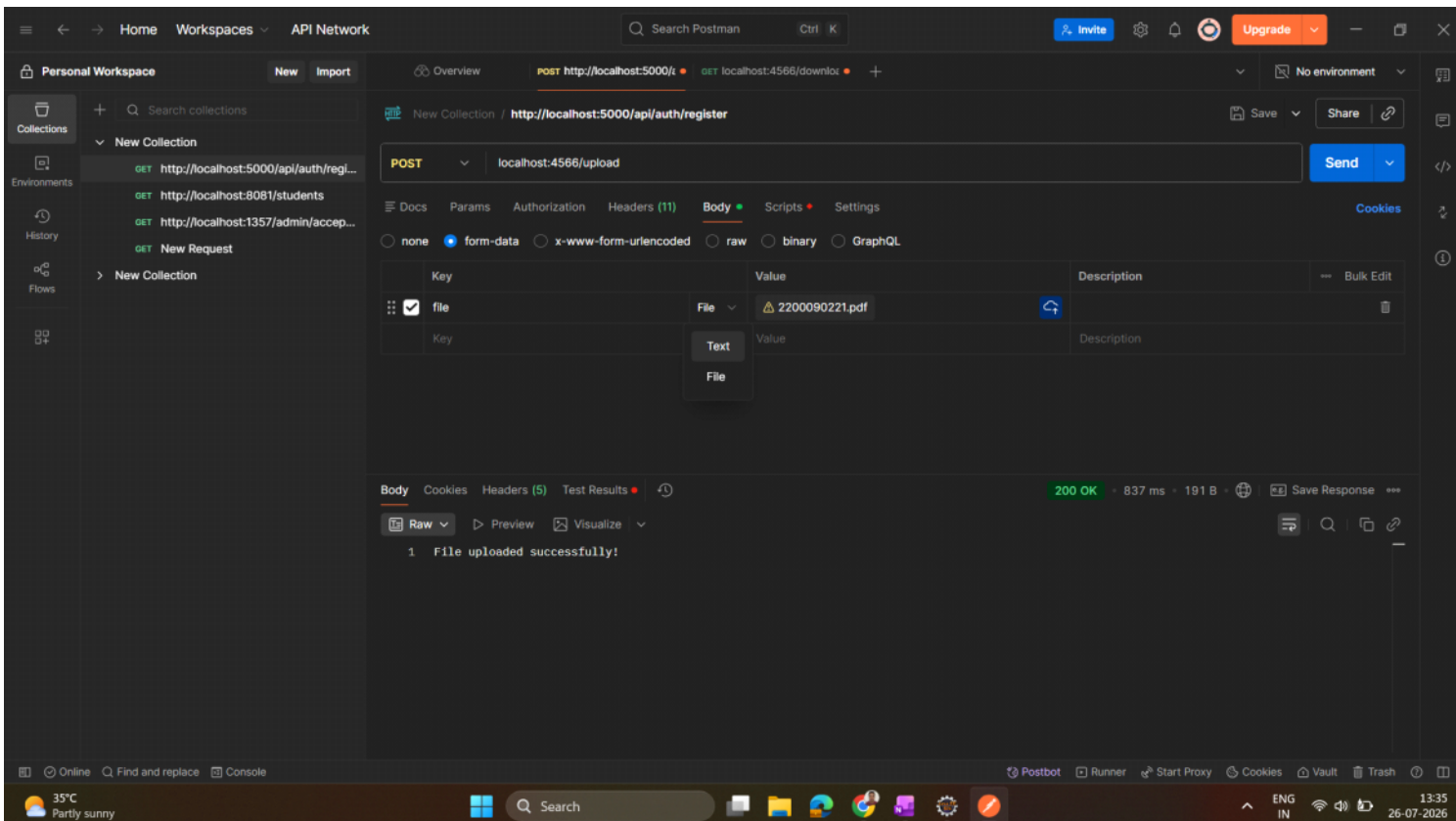
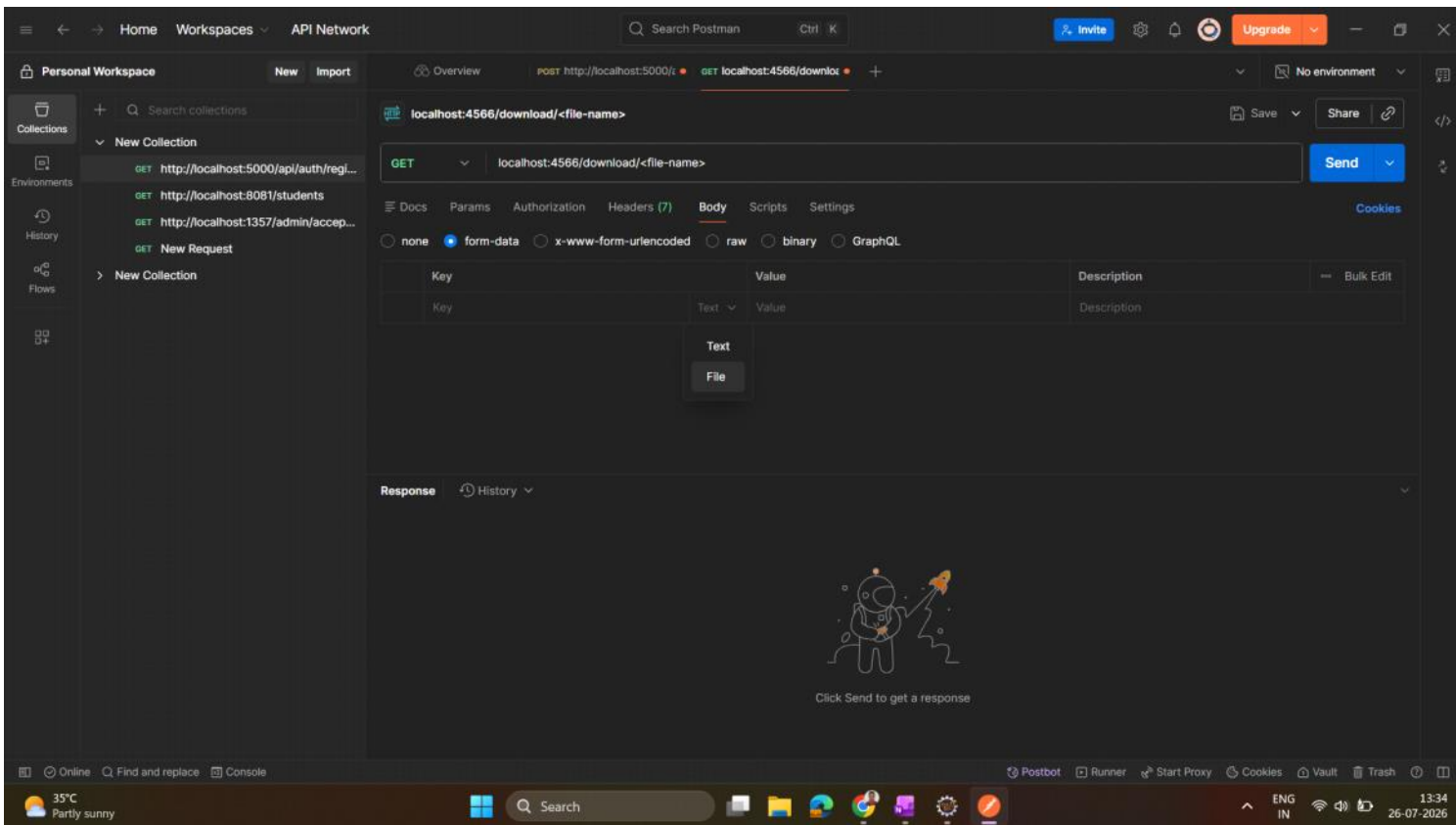
Step7: Go to the postman and run the links

```
localhost:XXXX/upload
localhost:XXXX/download/<file-name>
```

check if they are working good else check the code after getting the errors

Note:

Code must be <accesskey, secretkey> = don't reverse the order else will get the errors
The bucket name must not be in the " ", must be free ex: deepakch-bucket not "deepakch-bucket" else the eclipse ide wont understand the bucket existence and will raise errors.



Checking in postman and then go to S3bucket and see the uploads

Final Desired output:

Amazon S3

_buckets

General purpose buckets

Directory buckets

Table buckets

Vector buckets

Files

File systems

Access management and security

Access Points

Access Points for FSx

Access Grants

IAM Access Analyzer

Storage management and insights

Storage Lens

Batch Operations

springsnippet-s3-work

Objects (2)

Objects are the fundamental entities stored in Amazon S3. You can use [Amazon S3 inventory](#) to get a list of all objects in your bucket. For others to access your objects, you'll need to explicitly grant them permissions. [Learn more](#)

Find objects by prefix

Name	Type	Last modified	Size	Storage class
2200090221.pdf	pdf	July 26, 2026, 13:20:03 (UTC+05:30)	716.9 KB	Standard
Guideline - Capstone Project Final Review.pdf	pdf	July 26, 2026, 13:19:40 (UTC+05:30)	183.3 KB	Standard

CloudShell Agent Toolkit for AWS Feedback Console Mobile App

© 2026, Amazon Web Services, Inc. or its affiliates. Privacy Terms Cookie preferences

35°C Partly sunny

For postman the downloaded folder may not read properly so click send and download and save the file as filename.pdf and the users may view it

Personal Workspace

New Import

Overview **POST** http://localhost:5000/api/auth/register

New Collection / http://localhost:5000/api/auth/register

POST localhost:5000/upload

Docs Params Authorization Headers (11) **Body** Scripts Settings

none form-data x-www-form-urlencoded raw binary GraphQL

Key	Value	Description
file	2200090221.pdf	

Body Cookies Headers (5) Test Results

Raw Preview Visualize

1 File uploaded successfully!

200 OK 837 ms 101 B Save Response

Postbot Runner Start Proxy Cookies Vault Trash

35°C Partly sunny

Step9: Remove all the files>S3 bucket else will cost you

Step10: Successful Springboot backend connection with S3 bucket